Transaktionen verstehen

Acht forensische Ethereum-Fälle: Transaktionsformate, EVM, Zustandsänderungen und Security

8 FALLSTUDIEN

40 PUNKTE

LERNZIELE

- **Transaktionsformate Type 0 bis 4 sicher erkennen**
- **EVM-Ausführung, Revert und Gebühren logisch erklären**
- **Transaktionshistorie, Receipts und Ethereum-State sauber trennen**
- **Änderungen an Nonce, Balance, Code, Storage und Roots ableiten**
- **Sicherheitsrisiken und Fehlannahmen bei der Analyse erkennen**

ARBEITSWEISE

Bearbeiten Sie jeden Fall unabhängig. Halten Sie pro Aufgabe den Typ, das Ausführungsergebnis, die Datenstruktur-Änderungen, die konkreten State-Updates und einen Sicherheitsaspekt fest. Sofern nicht ausdrücklich anders angegeben, gelten gültige Signaturen, ausreichende Gebühren und erfolgreiche Blockaufnahme. Die Beispiele sind zur Lehre verkürzt und keine broadcastfähigen Rohtransaktionen.

NAME: _____

MATR.-NR.: _____

FORMAT

A4 · 8 Fälle · unabhängige Szenarien

01 ETH-Transfer an Bob

TYPE 0 | LEGACY · Niveau: Basis · 5 Punkte

SZENARIO

Ein einfacher ETH-Transfer ohne Smart-Contract-Code. Vergleichen Sie die Zustandsänderungen mit dem UTXO-Modell von Bitcoin.

AUSGANGSZUSTAND

Alice: 5 ETH, nonce 7. Bob: 1 ETH. Block-Base-Fee: 15 Gwei. Beide Adressen sind EOAs.

TRANSAKTION (DIDAKTISCH GEKÜRZT)

```
type: Legacy (kein Type-Präfix)
nonce: 7
to: Bob
value: 1 ETH
data: 0x
gasPrice: 20 Gwei
gasLimit: 21.000
gasUsed: 21.000
```

IHRE ANALYSE

01 Woran erkennen Sie das Legacy-Format?

02 Wie hoch ist die Gebühr und wie lauten beide ETH-Guthaben danach?

03 Welche Nonce ändert sich und warum?

04 Welche Datenstrukturen bzw. Roots ändern sich?

05 Nennen Sie einen Unterschied zum Bitcoin-UTXO-Modell.

NOTIZEN / RECHENWEG

02 Access List und SLOAD

TYPE 1 | EIP-2930 · Niveau: Mittel · 5 Punkte

SZENARIO

Ein Zugriff auf einen Contract-Speicherwert mit vorab deklarierter Access List; die Transaktion erzeugt keine persistenten Storage-Schreibvorgänge.

AUSGANGSZUSTAND

Der Counter-Contract hat Storage-Slot 0 = 42. getCount() liest nur Slot 0, schreibt nichts und emittiert kein Event.

TRANSAKTION (DIDAKTISCH GEKÜRZT)

```
type: 0x01
to: Counter
value: 0 ETH
data: ABI(getCount())
gasPrice: 20 Gwei
gasLimit: 80.000
gasUsed: 30.000
accessList: [Counter, Slot 0]
```

IHRE ANALYSE

01 Wodurch unterscheidet sich Type 1 von Type 0 und Type 2?

02 Warum ist eine Access List nützlich? Was garantiert sie nicht?

03 Ändert sich der Wert 42 oder der storageRoot des Counter?

04 Wird der Rückgabewert 42 im normalen Receipt abgelegt?

05 Welche Zustandsänderungen entstehen trotz rein lesendem Contract Call?

NOTIZEN / RECHENWEG

03 Neuen Contract deployen

TYPE 2 | CREATE · Niveau: Mittel · 5 Punkte

SZENARIO

Ein Deployment: Die Transaktion enthält Initcode, aber ein neuer Account erhält nur den nach Ausführung erzeugten Runtime-Code.

AUSGANGSZUSTAND

Alice hat nonce 10 und ausreichend ETH für Gas. Der Konstruktor setzt count = 7. Deployment ist erfolgreich.

TRANSAKTION (DIDAKTISCH GEKÜRZT)

```
type: 0x02
nonce: 10
to: null (Contract-Erstellung)
value: 0 ETH
data: Initcode + ABI(7)
gasLimit: 300.000
maxFeePerGas: 40 Gwei
maxPriorityFeePerGas: 2 Gwei
```

CONTRACT / AUSFÜHRUNG

```
contract Counter {
  uint256 public count;
  constructor(uint256 initial) {
    count = initial;
  }
}
```

IHRE ANALYSE

01 Wie erkennen Sie eine direkte Contract-Erstellung?

02 Wie wird die neue Contract-Adresse bestimmt?

03 Welche vier Account-Felder bekommt der neue Contract?

04 Wo liegt count = 7, wo der Runtime-Bytecode?

05 Was bleibt vom Initcode langfristig nachweisbar?

NOTIZEN / RECHENWEG

04 2 ETH im Vault einzahlen

TYPE 2 | CONTRACT CALL · Niveau: Mittel · 5 Punkte

SZENARIO

Ein payable-Funktionsaufruf ändert sowohl den ETH-Bestand eines Contracts als auch einen Wert in dessen Storage Trie.

AUSGANGSZUSTAND

Alice: 10 ETH, nonce 5. Vault: 5 ETH. deposits[Alice] = 1 ETH. Base Fee: 20 Gwei.

TRANSAKTION (DIDAKTISCH GEKÜRZT)

```
type: 0x02
nonce: 5
to: Vault
value: 2 ETH
data: 0xd0e30db0  [deposit()]
gasLimit: 100.000
gasUsed: 50.000
maxFeePerGas: 40 Gwei
maxPriorityFeePerGas: 3 Gwei
```

CONTRACT / AUSFÜHRUNG

```
mapping(address => uint256) deposits;
function deposit() external payable {
  deposits[msg.sender] += msg.value;
}
```

IHRE ANALYSE

01 Welche Funktion wird durch 0xd0e30db0 aufgerufen?

02 Wie viel ETH besitzen Alice und Vault danach (inkl. Gas)?

03 Wie lautet der neue Wert von deposits[Alice]?

04 Wie wird der Mapping-Storage-Slot hergeleitet?

05 Welche Account-Felder, Tries und Roots ändern sich?

NOTIZEN / RECHENWEG

05 100 ERC-20-Token senden

TYPE 2 | TOKEN CALL · Niveau: Mittel · 5 Punkte

SZENARIO

Eine ERC-20-Übertragung ist ein Smart-Contract-Aufruf, keine eigene Ethereum-Transaktionsformatklasse.

AUSGANGSZUSTAND

TOKEN: balances[Alice] = 130 TOKEN; balances[Bob] = 25 TOKEN. Die transfer()-Ausführung ist erfolgreich und emittiert Transfer.

TRANSAKTION (DIDAKTISCH GEKÜRZT)

```
type: 0x02
to: TOKEN
value: 0 ETH
data: 0xa9059cbb || ABI(Bob,100)
gasLimit: 100.000
Ausführung: transfer(Bob, 100)
```

CONTRACT / AUSFÜHRUNG

```
require(balances[msg.sender] >= amount);
balances[msg.sender] -= amount;
balances[to] += amount;
emit Transfer(msg.sender,to,amount);
```

IHRE ANALYSE

01 Welches Feld verrät den üblichen ERC-20 transfer()-Aufruf?

02 Warum steht value = 0 ETH?

03 Wie ändern sich die Token-Balances von Alice und Bob?

04 In welcher Datenstruktur stehen die Token-Salden und das Event?

05 Warum reicht ein 4-Byte-Selektor als Echtheitsbeweis nicht aus?

NOTIZEN / RECHENWEG

06 Withdraw revertiert

TYPE 2 | REVERT · Niveau: Fortgeschritten · 5 Punkte

SZENARIO

Eine gültige Transaktion kann erfolgreich in einen Block aufgenommen werden, obwohl ihre EVM-Ausführung fehlgeschlagen ist.

AUSGANGSZUSTAND

Vault speichert deposits[Alice] = 3 ETH. Alice versucht 5 ETH abzuheben. require() scheitert vor jeder Schreiboperation.

TRANSAKTION (DIDAKTISCH GEKÜRZT)

```
type: 0x02
to: Vault
value: 0 ETH
data: ABI(withdraw(5 ETH))
gasLimit: 100.000
gasUsed: 40.000
Ausführung: require(3 >= 5) -> false
```

CONTRACT / AUSFÜHRUNG

```
function withdraw(uint256 amount) external {
  require(deposits[msg.sender]>=amount);
  deposits[msg.sender]-=amount;
  (bool ok,) = msg.sender.call{value:amount}("");
  require(ok);
}
```

IHRE ANALYSE

01 Ist die Transaktion ungültig, fehlgeschlagen oder erfolgreich?

02 Wird ein Receipt erzeugt und welcher status steht darin?

03 Ändern sich Vault-Balance oder deposits[Alice]?

04 Wer bezahlt Gas und welche Nonce wird erhöht?

05 Kann der globale stateRoot trotz Revert anders sein?

NOTIZEN / RECHENWEG

07 Rollup veröffentlicht Blob

TYPE 3 | EIP-4844 · Niveau: Fortgeschritten · 5 Punkte

SZENARIO

Eine Blob-Transaktion verbindet eine normale Execution-Transaktion mit separat verbreiteten Daten für Data Availability.

AUSGANGSZUSTAND

RollupInbox.lastBatch = 41. postBatch(42) schreibt 42 und emittiert BatchPosted. Blob-Fee-Cap ist ausreichend.

TRANSAKTION (DIDAKTISCH GEKÜRZT)

```
type: 0x03
to: RollupInbox
data: ABI(postBatch(42))
blobVersionedHashes: [0x01ab...]
maxFeePerBlobGas: 10 Gwei
gasLimit: 150.000
Sidecar: Blob + KZG-Commitment + Proof
```

IHRE ANALYSE

01 Welche Felder kennzeichnen Type 3?

02 Enthält der Transaction Trie die vollständigen Blob-Daten?

03 Wie werden Blob und Versioned Hash kryptografisch verknüpft?

04 Was kann die EVM über Blobs direkt lesen?

05 Wo liegen Blobs und was gilt für ihre langfristige Verfügbarkeit?

NOTIZEN / RECHENWEG

08 EOA delegiert Smart-Wallet-Code

TYPE 4 | EIP-7702 · Niveau: Experte · 5 Punkte

SZENARIO

Eine EOA autorisiert vorhandenen Contract-Code. Ein Relayer reicht die Transaktion ein und bezahlt die Ausführung.

AUSGANGSZUSTAND

Alice: EOA ohne Code, nonce 4. Relayer: nonce 12. WalletLogic existiert. Die Autorisierung ist gültig und setLimit(5) gelingt.

TRANSAKTION (DIDAKTISCH GEKÜRZT)

```
type: 0x04
from*: Relayer; nonce: 12
to: Alice; data: ABI(setLimit(5))
authorizationList: [(chainId=1,
  WalletLogic, nonce=4, Sig_Alice)]
signed outer tx: Sig_Relayer
*from nur Analysehilfe, kein Feld
```

CONTRACT / AUSFÜHRUNG

```
contract WalletLogic {
  uint256 public limit;
  function setLimit(uint256 n) external {
    limit = n;
  }
}
```

IHRE ANALYSE

01 Woran erkennt man Type 4? Wozu zwei Signaturen?

02 Welcher Code wird bei Alice hinterlegt oder referenziert?

03 In wessen Account liegt limit = 5 nach dem Aufruf?

04 Wie ändern sich die Nonces von Alice und Relayer?

05 Welche neuen Risiken entstehen durch eine Delegation?

NOTIZEN / RECHENWEG

09 Vergleichsmatrix

Welche Datenstrukturen werden durch die acht konkreten Transaktionen verändert?

Tragen Sie J (ja), N (nein) oder U (nur unter Bedingungen) ein. Begründen Sie, weshalb ein Revert trotzdem den Global State verändern kann.

Fall	Tx Trie	Receipt Trie	State Trie	Storage Trie	Code
01					
02					
03					
04					
05					
06					
07					
08					

REFLEXION

Warum ist die Unterscheidung zwischen Transaktionshistorie und aktuellem Ethereum-Zustand für einen Security-Analysten wichtig?